

FHEStore: FPGA-Based Computational Storage for Data Preparation in Fully Homomorphic Encryption

Anonymous Author(s)

Abstract

Fully homomorphic encryption (FHE) allows computation directly on ciphertexts without decrypting the data. Existing FHE acceleration research focuses more on ciphertext computation, while the complete data-preparation path from stored data to ready-to-use ciphertext inputs receives comparatively less attention. For applications processing stored data, the conventional path relies on the host CPU to read plaintext and perform encoding, encryption, and ciphertext organization. As homomorphic computation is accelerated, the relative overhead of this data-preparation stage becomes more prominent; plaintext-to-ciphertext expansion also increases buffering requirements and data movement.

We present FHEStore, which uses FPGA-based computational storage to connect data access, optional preprocessing, and encoding and encryption into a storage-side data-preparation path, generating ciphertext inputs for subsequent FHE computation. Its high-level synthesis (HLS) operator combines radix encoding with Fast Fully Homomorphic Encryption over the Torus (TFHE) encryption. Each generated mask coefficient feeds both dot-product accumulation and incremental ciphertext output, avoiding buffering a complete mask vector. An operator stream composition method further accounts for ciphertext expansion by determining output buffer capacity from the input value count and ciphertext representation and coordinating preprocessing and encryption operators through stream flow control.

The evaluation compares the FHEStore prototype with host CPU baselines. With 1 KiB of plaintext input, the encryption operator achieves a $5.00\times$ speedup, while data preparation achieves a $2.45\times$ speedup. The evaluation also shows that FHEStore reduces cumulative host CPU time and peak host memory usage.

CCS Concepts

• **Hardware** → **Hardware accelerators**; • **Information systems** → **Storage architectures**; • **Security and privacy** → **Cryptography**.

Keywords

Fully Homomorphic Encryption, TFHE, FPGA, High-Level Synthesis, Computational Storage, Streaming Dataflow

ACM Reference Format:

Anonymous Author(s). 2027. FHEStore: FPGA-Based Computational Storage for Data Preparation in Fully Homomorphic Encryption. In *Proceedings of 35th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '27)*. ACM, New York, NY, USA, 11 pages.

1 Introduction

Fully homomorphic encryption (FHE) enables a data owner to outsource computation without revealing the plaintext to the evalu-

FPGA '27, California, USA

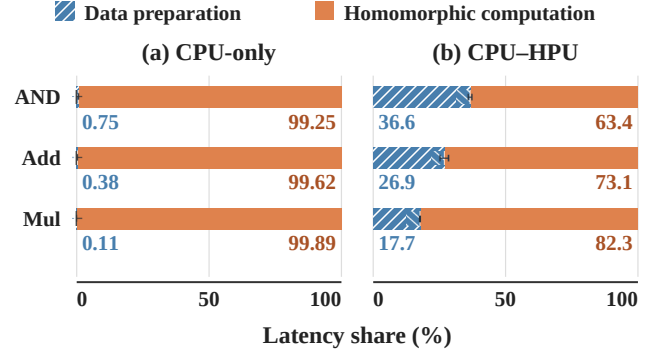


Figure 1: Latency shares for (a) CPU-only and (b) CPU-HPU execution. Both use CPU-based data preparation.

ator [4]. Hardware accelerators improve homomorphic computation through specialized arithmetic and memory-aware architectures [8, 9, 18, 19, 24, 28, 31]. For applications processing stored data, the conventional path still requires the host CPU to read plaintext and perform encoding, encryption, and ciphertext organization before homomorphic computation. We refer to the process from persistent-plaintext access to ciphertext availability in host memory as data preparation. In terms of ciphertext generation and use, data preparation constitutes the ciphertext-production stage, while homomorphic computation constitutes the ciphertext-consumption stage. Application performance therefore depends on both data preparation and homomorphic computation.

Figure 1 compares bitwise AND, addition, and multiplication on a CPU and Zama’s open-source FPGA-based homomorphic processing unit (HPU) [30]. Both workflows use the same host CPU data preparation: SSD reads, encoding, and encryption, ending with ciphertexts in host memory. They differ only in the homomorphic-computation stage, using CPU software execution and HPU acceleration, respectively. Stage shares are normalized to the sum of data-preparation and homomorphic-computation latencies, excluding network transfer; bars show means, and error bars span the observed minimum and maximum. With the data-preparation path unchanged, homomorphic acceleration increases the data-preparation latency share from 0.11%–0.75% to 17.7%–36.6%, making host-side data preparation an increasingly important optimization target.

Host-side data preparation requires not only encoding and encryption but also coordination of data access and ciphertext output. Encoding and encryption consume CPU resources, while plaintext-to-ciphertext expansion increases output buffering requirements and data movement. Optional preprocessing, such as record selection and field projection, can reduce the data to be processed before encryption but must also be connected to subsequent encoding and encryption. Optimizing the complete data-preparation path

therefore requires both accelerating cryptographic operations and coordinating storage access, preprocessing, and expanded ciphertext output.

Existing encryption accelerators reduce encryption overhead through specialized cryptographic arithmetic and parallel execution [3, 11, 12, 22], while client-side and end-to-end systems further coordinate encoding, memory organization, and cryptographic processing [6, 10, 25, 29]. For stored data, the complete data-preparation path also involves persistent-data access, optional preprocessing, and ciphertext output management. Accelerating encryption alone does not directly address coordination across these stages. Computational storage places programmable processing near storage access, providing a basis for organizing this complete path [17, 20, 26, 27].

We present FHEStore, which uses FPGA-based computational storage to connect data access, optional preprocessing, and encoding and encryption into a storage-side data-preparation path, generating ciphertext inputs for subsequent FHE computation. Its high-level synthesis (HLS) encryption operator integrates radix encoding with Fast Fully Homomorphic Encryption over the Torus (TFHE) encryption [2]. Each generated mask coefficient feeds both dot-product accumulation and incremental ciphertext output, avoiding buffering a complete mask vector. Its operator stream composition method connects optional preprocessing to encryption, determines output buffer capacity from the input value count and ciphertext representation, and coordinates the composed operators through stream flow control. The host submits tasks and retrieves the generated ciphertexts, supplying inputs for subsequent homomorphic computation.

The main contributions are as follows:

- **Integrated HLS operator for TFHE encoding and encryption.** A TFHE encryption operator integrates radix encoding and encryption, traversing the mask once per learning-with-errors (LWE) block. Each coefficient feeds dot-product accumulation and incremental ciphertext output without buffering a complete mask vector.
- **Operator stream composition accounting for ciphertext expansion.** A method connects optional preprocessing to encryption, determines ciphertext buffer capacity from the value count and ciphertext representation, and coordinates the composed operators through stream flow control.
- **Prototype implementation and evaluation.** The evaluation compares the FHEStore prototype with host CPU baselines (Section 6). With 1 KiB of plaintext input, the encryption operator and data preparation achieve 5.00× and 2.45× speedups, respectively, with lower cumulative host CPU time and peak host memory usage.

2 Background

2.1 FHE and TFHE Fundamentals

FHE enables computation on encrypted data without revealing the underlying plaintext to the evaluator [4]. It provides key generation, encryption, homomorphic evaluation, and decryption. TFHE is a lattice-based FHE scheme over the torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$, where the period is normalized to one [2].

Encoding maps application values into the scheme’s message representation; decoding maps decrypted messages back into application values. Let k_{enc} , k_{dec} , and k_{eval} denote the encryption, decryption, and evaluation key material. For messages $m_1, \dots, m_t$, form $c_i \leftarrow \text{Enc}_{k_{\text{enc}}}(m_i)$. Evaluation applies f to these ciphertexts using k_{eval} to obtain c_f . With valid parameters, an exact scheme satisfies the following relation for a supported computation f , with overwhelming probability over its randomness:

$$\text{Dec}_{k_{\text{dec}}}(c_f) = f(m_1, \dots, m_t). \quad (1)$$

LWE describes noisy linear equations modulo an integer, providing a basis for lattice-based cryptography [16]. TFHE uses LWE and ring learning-with-errors (RLWE) ciphertexts, with bootstrapping to control noise [2]. For secret-key torus-LWE encryption, let $\mu = \text{Encode}(m)$ belong to a discrete message space $\mathcal{M} \subset \mathbb{T}$, and let $\mathbf{s} \in \mathbb{Z}^n$ be the secret key. Sampling a uniform mask $\mathbf{a} \in \mathbb{T}^n$ and an error e from the scheme’s noise distribution yields

$$c = (\mathbf{a}, b), \quad b = \langle \mathbf{a}, \mathbf{s} \rangle + \mu + e \pmod{1}. \quad (2)$$

Decryption computes the phase,

$$\phi_s(c) = b - \langle \mathbf{a}, \mathbf{s} \rangle = \mu + e \pmod{1}, \quad (3)$$

and recovers the encoded message when the phase remains within its decision region [2].

A finite-precision representation stores each torus coefficient as a q -bit integer corresponding to a multiple of 2^{-q} . Torus addition then becomes integer addition modulo 2^q . This representation connects the encryption equation to coefficient arithmetic in hardware.

For integer inputs, radix encoding decomposes each application value into fixed-width digits. Each digit is encoded as a torus message and encrypted as an LWE block; the ordered blocks form a ciphertext bundle representing that value. Each block contains $n+1$ coefficients, so the bundle’s size depends on the digit count, LWE dimension, and coefficient precision. The value count and ciphertext bundle size determine the ciphertext buffer capacity and transfer volume required during data preparation.

2.2 FHE Producer–Consumer Workflows

Outsourced FHE systems place data preparation in the data owner’s trusted domain and homomorphic computation at an external consumer [6, 10, 29]. The producer coordinates data access, encoding, encryption, and ciphertext organization. The consumer executes the encrypted program using ciphertexts and evaluation material; the secret key remains in the trusted domain.

Client-side accelerators connect encoding and encryption to software control and memory transfers [10, 29], while FHE accelerators schedule encrypted operations over ciphertexts and evaluation keys in their memory systems [24, 28, 31]. Producer and consumer must agree on cryptographic parameters, message encoding, and ciphertext layout. Client-side systems coordinate local processing with encrypted computation across this interface [25].

2.3 Computational Storage and Application Integration

Computational storage couples processing with storage to offload host work or reduce data movement [20]. Operations such as filtering, aggregation, feature selection, and decompression transform

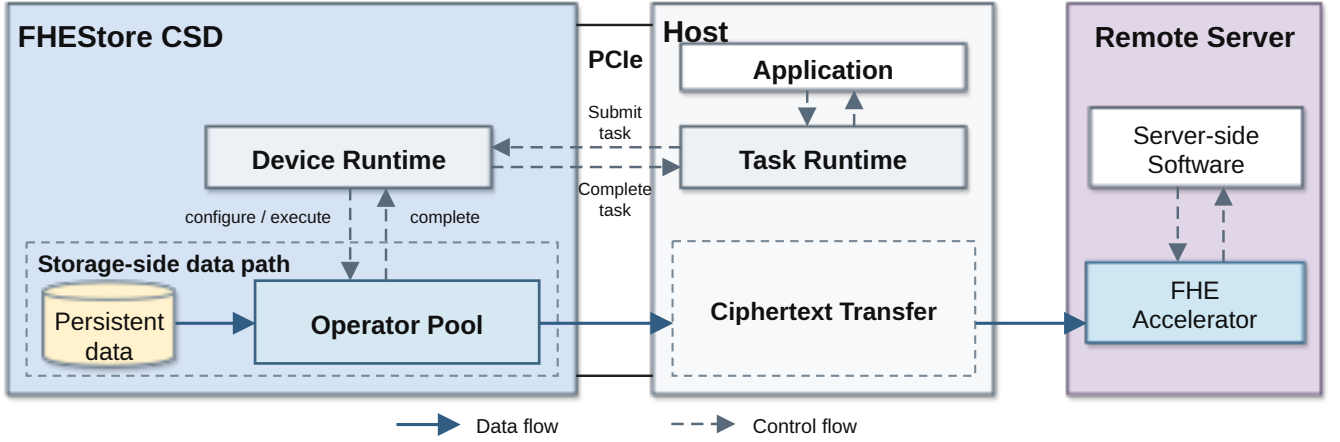


Figure 2: FHEStore architecture for FHE data preparation. Solid blue arrows denote data flow; dashed gray arrows denote control flow.

data along the storage-access path [17]. Their output representation affects the benefit: filtering can shrink the returned data, while encryption expands it.

FPGA storage frameworks expose data access and processing through virtual files, C++ data and parameter streams, and unified HLS interfaces for memory and storage [17, 26]. These abstractions express task parameters and input and output locations while the framework coordinates transfers and completion. FHEStore integrates encryption with this storage-side execution model (Section 3).

3 System Overview

3.1 System Organization

FHEStore organizes data preparation across a CSD and host software connected over PCIe (Figure 2). The CSD combines persistent storage, device-local buffers in Subsystem Local Memory (SLM) [13], and an FPGA operator pool. The remote server represents an external privacy-preserving outsourcing service equipped with an FHE accelerator.

Persistent data enter device-local buffers and feed an operator graph whose modules exchange records or values through streams. Optional preprocessing supplies application values to the encryption operator, which integrates encoding with encryption. Generated ciphertexts are collected in device-local output buffers.

The host task runtime interprets application requests, while the device runtime binds them to the CSD’s operators and memory resources. Control messages submit an execution program describing operator requirements and stream connections, together with operator contexts, then report completion. Ciphertext transfer retrieves the output SLM range into host memory over PCIe and identifies the ciphertext sequence using the reported valid ciphertext length. The host can then forward the sequence over the network to the external consumer.

As in Section 2, the host and CSD belong to the data owner’s trusted domain, where plaintext processing and key-dependent encryption occur. The host resolves locally held key material and

supplies the LWE secret key s through the encryption operator’s context when preparing execution. The external consumer receives ciphertexts, while local task control retains their association with the source data.

3.2 Application Task Model

An application specifies a data-preparation task $T = (D, Q, E, O)$. D identifies the stored input and its layout, Q specifies optional preprocessing, E selects the encoding and cryptographic configuration, and O identifies the output location in host memory. The task runtime checks these descriptions before submitting device execution.

The input description combines a storage range with its content interpretation. With Q omitted, packed application values enter encryption directly in source order; otherwise, preprocessing supplies the values to encrypt. Both forms define an ordered sequence $x_0, \dots, x_{M-1}$ of M application values. With configuration E and secret key s , the task produces

$$c_j \leftarrow \text{Enc}_{s,E}(\text{Encode}_E(x_j)), \quad 0 \leq j < M. \quad (4)$$

Here, c_j is the ciphertext bundle representing x_j under E , including multiple ciphertexts when required by the encoding. The task returns the ordered ciphertext sequence at O , independently of the device resources used.

Selection and projection provide one case of optional preprocessing. For source records $r_0, \dots, r_{N-1}$, the schema in D identifies record boundaries, field types, and offsets. Selection evaluates the predicates in Q , and projection chooses the field passed to encryption. Let $I_Q = (i_0, \dots, i_{M-1})$ list the selected indices in source order and π_Q denote the field projection. Equation (4) then uses $x_j = \pi_Q(r_{i_j})$.

For a selective task, the host retains the source-record mapping I_Q derived from the selection manifest, which records source-record identities and selection results. This mapping associates each ciphertext bundle with its source record, including records with identical projected values. An empty selection yields an empty ciphertext sequence and mapping.

3.3 Operator Graphs and Device Execution

The task runtime translates T into an operator graph, operator contexts, and input and output SLM ranges. Graph nodes identify operator functions, and edges specify the streams exchanged between their ports. An execution program records the graph's operator requirements, connections, and entry and terminal endpoints for submission to the device runtime. Keeping these objects distinct connects application semantics to device execution: the graph defines the computation, contexts supply its task-specific interpretation, and SLM ranges identify the data it consumes and produces.

Operator contexts carry the input interpretation from D , optional preprocessing settings from Q , and the encryption configuration from E . The encryption operator's context also contains its locally supplied key material. Contexts bind task-specific parameters to the graph's logical operators. For a selective task, the operator context for preprocessing determines which fields are inspected and which field is emitted, while the operator context for encryption determines how each emitted value becomes a ciphertext bundle. Section 4 develops the encryption operator's internal design.

An input SLM range identifies the device-local input buffer, its offset, and its byte extent. An output SLM range identifies the destination buffer and the extent available for ciphertext reception. These ranges bind graph endpoints to data locations without encoding those locations into the operator's processing logic. Their extents also distinguish the bounded input consumed by an execution from the capacity reserved for its expanded output. The device runtime receives the range bindings together with the execution program and operator contexts.

Before execution, the device runtime assigns the graph to available operators, installs their contexts, and binds stream destinations through the stream switch. The graph's logical connections determine the downstream operator or output endpoint for each stream. A task without preprocessing uses the encryption operator alone; a selective task connects preprocessing to encryption. Connected operators exchange values through the operator interconnect during device execution.

Device completion reports the valid ciphertext length in the output SLM range after the graph finishes, separately from allocated capacity and execution framing. The task runtime passes this length to ciphertext transfer. Application completion denotes availability at O in host memory, with the source-record mapping retained for selective tasks. Section 5 develops the operator stream composition method, including stream routing and ciphertext buffer allocation.

4 HLS Design of TFHE Encoding and Encryption

FHEStore's HLS encryption operator receives ordered application values and generates their ciphertext bundles. Figure 3 organizes the datapath into input and context, sampling, encryption arithmetic, and packetization, connecting the torus-LWE operation in Section 2 to the storage-side stream in Section 3. Encoding and encryption share one invocation: radix digits become LWE blocks within the datapath, while coefficients leave incrementally through the packetizer.

4.1 Representation and Streaming Interface

Let w denote the application-value width, p the message bits per radix digit, and $K = \lceil w/p \rceil$ the number of digits representing a value. Each digit produces one LWE block with n mask coefficients and a body. Thus, a ciphertext bundle contains K blocks, ordered from the least-significant digit to the most-significant digit. Torus coefficients have width q , while the physical stream has width W . A beat is one transfer on this stream interface. The packetizer places $L = W/q$ coefficients in a ciphertext beat, with q dividing W . Widths and radix counts describe the synthesized datapath; the operator context provides the configuration for a particular execution.

The CPU coefficient layout serializes each block in natural order as $(a_0, \dots, a_{n-1}, b)$. This order gives a direct relationship between the mask index, key index, and output position. The arithmetic uses unsigned coefficients modulo 2^q , implementing the finite-precision form of torus addition in Equation (2). The binary secret key contains n bits, with $s_i \in \{0, 1\}$. Consequently, multiplication by a key coefficient becomes selection of a_i or zero, followed by modular addition.

Both physical ports use AXI4-Stream. The input adapter interprets application-value framing; the packetizer constructs ciphertext beats. Configuration and the bit-packed secret key arrive through the operator context stored in BRAM. Configuration includes the input mode, input count, and pseudorandom number generator (PRNG) seed and nonce. Key indexing selects a context word and a bit within that word. The same key supplies successive blocks, while the coefficient index restarts at each block boundary. The encryption operator reads execution configuration before consuming its first payload beat. The input count controls application-value consumption, whereas coefficient and digit counters control block generation; these counts remain separate as encryption expands each accepted value into a bundle.

Each block's $n+1$ coefficients occupy $\lceil (n+1)/L \rceil$ ciphertext beats, including padding in unused positions. The serialized ciphertext bundle size in bytes is

$$C = K \left\lceil \frac{n+1}{L} \right\rceil \frac{W}{8}. \quad (5)$$

Block padding belongs to this representation; a separate terminal beat follows the payload and carries completion metadata.

4.2 Integrated Encoding and Encryption

The input adapter supplies one application value x to the radix encoder at a time. Packed mode carries consecutive values within a physical stream beat. The adapter inspects the AXI4-Stream byte-enable mask, TKEEP, and extracts the valid values in stream order. A configured input count bounds consumption when the final physical beat contains additional data. The adapter retains the received beat while its values are processed, allowing the encryption core to operate on a narrower value interface than the external stream.

Scalar mode instead assigns one application value to each payload beat, using its low w bits. This framing matches a preprocessing operator that emits one projected field per selected record. Here, the beat itself identifies one value; the byte-enable mask does not specify a collection of packed inputs. Both modes present the same

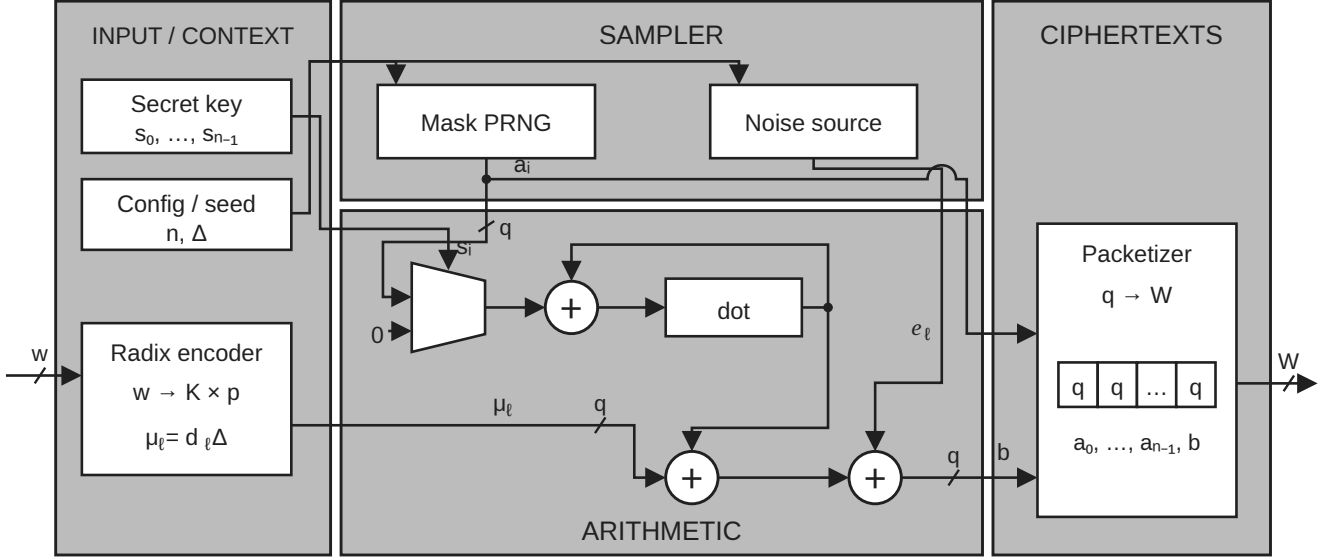


Figure 3: HLS architecture for integrated TFHE encoding and encryption.

value interface to the encoder, so the subsequent ciphertext bundle has the same digit and coefficient order.

For an unsigned w -bit value x , the radix encoder extracts digits and maps them to integer-torus messages:

$$\begin{aligned} d_\ell &= \lfloor x/2^{p\ell} \rfloor \bmod 2^p, \quad 0 \leq \ell < K, \\ \mu_\ell &= d_\ell \Delta. \end{aligned} \quad (6)$$

The encoding interval Δ places each digit within the message space represented by a coefficient. With a power-of-two interval, encoding consists of a shift into the corresponding coefficient positions. In this representation, the highest digit carries the remaining high-order bits when p does not divide w . The encoder produces each μ_ℓ inside the operator, without materializing an intermediate array of encoded torus messages.

The K digits reuse one encryption core sequentially. The core completes a block and its serialization before advancing to the next digit, and completes the bundle before accepting another value from the input adapter. Value and digit indices establish a ciphertext index for the mask PRNG, while the mask-coefficient index establishes the key position. This nested execution preserves the application-value order across both input modes and the least-significant-first order within each bundle.

4.3 Coefficient Arithmetic and State

The arithmetic core traverses each block's mask once. At coefficient index i , the PRNG generates a_i and the key reader supplies s_i . The same coefficient feeds the packetizer and binary-key selector, whose contribution updates the accumulator:

$$u_{i+1} = (u_i + s_i a_i) \bmod 2^q, \quad u_0 = 0. \quad (7)$$

After the traversal, the body $(u_n + \mu_\ell + e_\ell) \bmod 2^q$ follows the mask into the packetizer. Figure 3 shows the additions combining the dot product, digit, and noise.

The mask PRNG initializes its state from the context's seed and nonce and the ciphertext index. After mask generation, the noise source uses the same generator to obtain a bounded magnitude and sign for e_ℓ .

Incremental emission avoids retaining a complete mask vector before serialization. The datapath holds generator state, the dot-product accumulator, coefficient and digit indices, and a partially filled ciphertext beat; the packed key remains in the context. Conditional modular addition updates the accumulator, and body additions follow the completed recurrence. The W -bit output interface combines consecutive coefficients into a beat, while the arithmetic core generates those coefficients sequentially.

4.4 Ciphertext Packetization and Completion

The packetizer maintains a W -bit register and a count of occupied coefficient positions. Each arriving mask coefficient fills the next position. A full register is emitted as a ciphertext beat, after which the register and position count reset. The body follows the mask in the same sequence and forces a flush, even when fewer than L coefficient positions are occupied. Unused positions remain zero. The following LWE block starts in a fresh beat, preserving a block stride of $\lceil (n+1)/L \rceil$ beats and making block boundaries independent of input framing.

AXI4-Stream ready/valid flow control connects packetization to the surrounding data path. A blocked output write holds the pending beat and stalls progression through the coefficient traversal. The retained arithmetic and packet state preserve the relationship between generated coefficients, their accumulated contributions, and their output positions. The input adapter consequently waits while the core finishes the current ciphertext bundle.

The operator accepts a fixed input count or a stream whose length is determined by an upstream terminal beat. With a fixed count, it encrypts the requested values and waits for the terminal

beat; a premature terminal beat identifies an incomplete input. The terminal beat follows all ciphertext payload and carries completion metadata separately. Payload beats remain data beats regardless of a block boundary. Separating the terminal beat from block padding lets the receive path derive the valid ciphertext length and report device completion after ciphertext reception.

5 Storage-Side FHE Data Preparation

FHEStore’s operator stream composition method connects optional preprocessing to encryption while accounting for ciphertext expansion. It combines stream routing with ciphertext buffer allocation and stream flow control. Figure 4 shows the operator interconnect through which the composed operators exchange data.

5.1 Operator Stream Composition

The device runtime first loads the task’s SSD range into its input SLM range. Once loading completes, the read DMA supplies this range as an AXI4-Stream to the graph’s entry. The write DMA receives the final stream into an output SLM range, collecting ciphertexts as encryption produces them.

Memory binding and stream routing determine different parts of this path. DMA descriptors specify the buffer addresses and byte extents accessed at the memory endpoints. Destination tags identify where a beat travels within the operator graph. An operator-to-operator handoff therefore requires a downstream destination, while the memory endpoint requires a bound range in which to read or write data. This separation lets the same operator graph execute over different bound memory ranges.

The stream switch forwards beats according to the AXI4-Stream destination field, TDEST. Figure 4 illustrates route composition using slot and port identifiers: the upper and lower fields identify a destination slot and input port. The example route $\{3, 1\}$ selects Port 1 of Operator #3, while the reserved output route $\{F, 2\}$ selects memory-output channel 2. The device runtime’s scheduler binds graph edges to their destinations and configures each source operator’s next-hop route. Operator controllers attach the resulting destination tag to outgoing beats, while the device runtime programs the read DMA’s entry destination. Each operator returns its output to the same stream switch, which forwards it to the next operator or the write DMA.

For packed values, the graph connects read DMA directly to encryption and routes ciphertext bundles to write DMA. Optional preprocessing inserts operators before encryption while retaining the graph’s memory endpoints.

For a selective task, the selection and projection operator implements Q from Section 3.2. Its context specifies record layout, predicate-field offsets and types, comparison operands, Boolean expression, and projected field. It captures required fields as record beats arrive and evaluates predicates at record boundaries. These settings can change independently of stream destinations.

During the selective task’s encryption pass, each matching record supplies one projected value in source order to the encryption operator’s scalar interface (Section 4.2). Values pass through the stream switch without an intermediate array in SLM, and ciphertext bundles follow the direct graph’s output route.

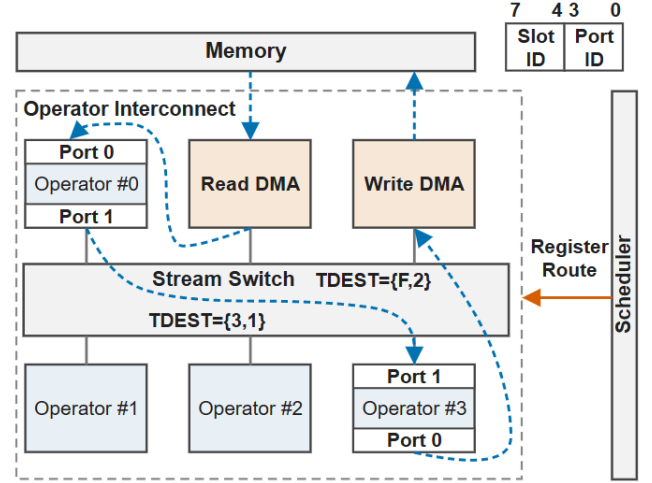


Figure 4: Operator interconnect and an example stream route. Dashed blue arrows show data flow.

Ciphertext expansion couples flow control to operator composition: encryption may occupy many output beats for each input value. Input FIFOs absorb short rate differences, while AXI4-Stream backpressure stalls upstream processing when downstream capacity is unavailable. This mechanism coordinates preprocessing, encryption, and ciphertext reception within finite buffers.

5.2 Execution and Output Management

The output SLM range must accommodate the representation produced by the graph. For the M application values defined in Section 3.2, the ciphertext payload occupies MC bytes, where C is the serialized ciphertext bundle size in Equation (5). The output extent therefore depends on both the value count and the encoding configuration. The receive range reserves space for the payload and a terminal beat, rounded to the allocation alignment:

$$B_{\text{out}} = A \left\lceil \frac{MC + F}{A} \right\rceil, \quad (8)$$

where A is the allocation alignment and F is the terminal-beat size. Valid ciphertext length is MC ; terminal-beat and allocation padding bytes belong only to the receive extent. Without selection, M follows from the input count.

Because input framing leaves C unchanged, the task runtime sizes the receive range using M and C , independently of intermediate routes.

Execution begins by preparing the write DMA’s receive descriptors before starting the read DMA. This ordering makes the bound output range available when the operator graph starts producing ciphertexts. After the payload, the encryption operator emits a separate terminal beat. The device runtime recognizes the terminal beat in receive completion, excludes its bytes from payload accounting, and reports the valid ciphertext length. Device completion identifies the point at which the graph’s ciphertext output is available in SLM. Ciphertext transfer retrieves the aligned output range into host memory and identifies the valid ciphertext sequence using the

reported length, including serialized block padding and excluding the terminal beat and allocation padding.

For a selective task, FHEStore determines the data-dependent M through a discovery pass over the input SLM range. In this mode, preprocessing produces a selection manifest with an entry for each source record, containing its index, selection flag, and selected projected value, followed by a count summary. The stream switch directs this output to the write DMA and a temporary output SLM range. Its extent is determined by the source-record count, so the selection manifest can be received before the selected count is known. The host retrieves the selection manifest and checks its record identities, selection flags, task metadata, and summary count. It then obtains M and the source-record mapping $\mathcal{I}_Q$.

The task runtime uses M to allocate the ciphertext output SLM range and launches the encryption pass with the operator graph that connects preprocessing to encryption. This pass reads the same input SLM contents and applies the same selection and projection. For $M > 0$, the task uses one SSD load and two SLM scans. The selection manifest remains host-side task metadata, and the host associates the retrieved ciphertext bundles with $\mathcal{I}_Q$. If $M = 0$, the task skips the encryption pass and returns an empty ciphertext sequence and mapping.

Larger selective tasks use batches with bounded source-record counts, bounding input buffers, manifests, and maximum ciphertext output even when every record matches. Each batch runs the discovery pass, allocates ciphertext buffers, and runs the encryption pass when its selected count is nonzero.

Batches execute sequentially, releasing device buffers after output retrieval. The host adjusts selected indices by each batch's source-record offset, preserving source-record mapping and ciphertext order across batch boundaries.

6 Experimental Evaluation

6.1 Experimental Setup

We implement FHEStore on a SoC FPGA-based computational storage platform and evaluate FHE data preparation from persistent data. The evaluation covers encryption operator performance, data-preparation performance, host resource consumption, and application-directed preprocessing. The host software path serves as the baseline; figures and tables identify the two paths as CPU baseline and FHEStore.

Table 1 summarizes the platform configuration. The device-side ARM processor runs the management software for SSD access, device memory management, and FPGA task scheduling, while the FPGA programmable logic executes preprocessing and encryption operators. The computational storage client submits tasks to the device and retrieves ciphertexts. It then transfers the ciphertexts over the network to a remote server equipped with a Zama HPU, which performs homomorphic computation.

The FPGA design is synthesized using Vivado 2020.2, and the encryption operator runs at 250 MHz. Table 2 summarizes FHEStore's FPGA resource usage, covering computational operators, storage access, data movement, interconnects, and control logic, based on the hierarchical synthesis report.

The inputs are unsigned 8-bit integers (u8), each represented by four 2-bit radix blocks, with one LWE ciphertext per block.

Table 1: Experimental platform configuration.

Component	Configuration
Host CPU	Intel Xeon Gold 6248R
Host memory	256 GB DDR4
Host kernel	Linux 5.15.135-yfl
SoC FPGA platform	Fidus Sidewinder-100 with Zynq UltraScale+ XCZU19EG
Device processor	Quad-core ARM Cortex-A53
Device memory	16 GB
Device kernel	Linux 5.4.0
Persistent storage	NVMe SSD, SSSTC CA5-8D512x1, 512 GB
QEMU	KVM acceleration, 4 vCPUs, 16 GiB memory
QEMU guest kernel	Linux 5.4.211
Remote HPU	Zama HPU on V80

Table 2: FHEStore FPGA resource usage.

Resource	Used	Utilization
LUT	266,538	50.99%
FF	343,422	32.85%
LUTRAM	21,750	13.49%
BRAM (36 Kb equivalent)	577	58.64%
URAM	58	45.31%
DSP	10	0.51%

The LWE dimension is 2048, and coefficients are 64 bits wide. The encryption performance comparison uses the same 64-byte-padded ciphertext layout for both paths. Plaintext size is measured in bytes, with one byte per unsigned 8-bit input value.

The HPU experiments cover bitwise OR, bitwise AND, equality, right shift, addition, subtraction, and multiplication. Application-directed data preparation is evaluated using lineitem data generated by TPC-H dbgen [23] and the public Intel Lab sensor dataset [1], examining the combined execution of record selection, field projection, and encryption.

We evaluate encryption latency, data-preparation latency, cumulative host CPU time, and peak host memory. In all experiments, the host-side programs for both the CPU baseline and FHEStore run in the same KVM-accelerated QEMU guest described in Table 1. All performance experiments exclude warm-up runs and use ten measurements per configuration; reported values are arithmetic means. Correctness is verified against plaintext reference results.

6.2 Encryption Operator Performance

This subsection compares encryption-stage performance between the CPU baseline and FHEStore for plaintext sizes of 1–4096 bytes. FHEStore uses fixed-length binary records converted from TPC-H lineitem data as the plaintext source. Each byte is encrypted as an independent 8-bit plaintext integer, without record selection or field projection.

For the hardware-counter-based comparisons, the CPU baseline records encryption-function execution time, while FHEStore's

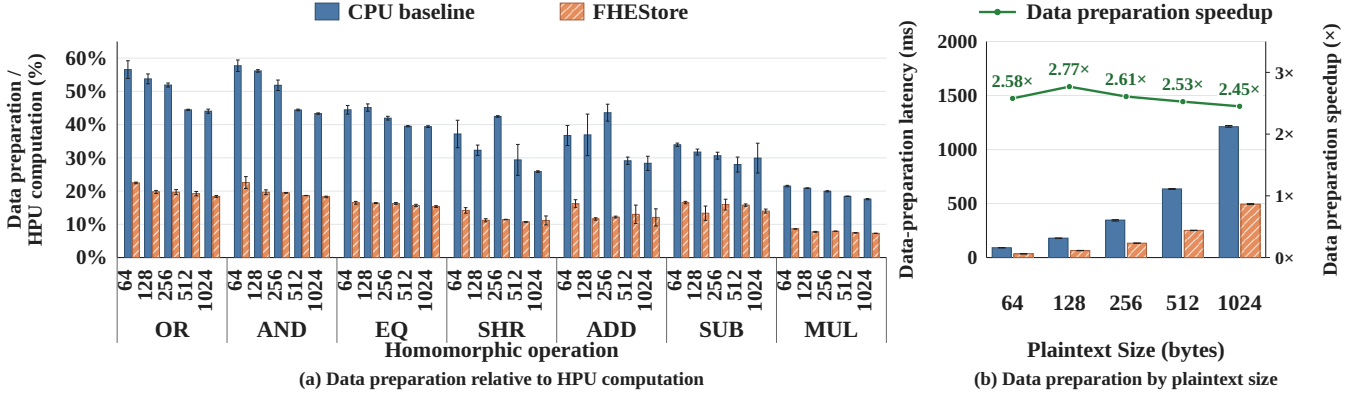


Figure 5: Data preparation: (a) latency relative to HPU computation and (b) latency by plaintext size, with speedup on the right axis. Plaintext size is measured per operand. Error bars show (a) run-level sample standard deviations and (b) ranges across seven operations.

Table 3: Encryption latency and speedup of FHEStore over the CPU baseline.

Plaintext Size (bytes)	CPU baseline (ms)	FHEStore (ms)	Speedup
1	0.597	0.570	1.05×
16	8.386	2.443	3.43×
32	15.518	3.659	4.24×
64	31.054	6.900	4.50×
128	62.289	13.091	4.76×
256	124.640	26.008	4.79×
512	252.330	51.261	4.92×
1024	507.577	101.596	5.00×
2048	1016.276	202.442	5.02×
4096	2197.291	411.879	5.33×

device-task time is obtained by converting hardware-counter cycles using the operator clock frequency and includes stream-interface waiting during task execution. Speedup is defined as

$$S_b = \frac{T_{\text{host},b}}{T_{\text{FHEStore},b}}, \quad (9)$$

where b is the plaintext size in bytes. The numerator and denominator are the CPU baseline and FHEStore latencies in Table 3, respectively.

Table 3 reports encryption latency and speedup across plaintext sizes. At 1 byte, the execution times are close, with a speedup of 1.05×. FHEStore achieves a 4.76× speedup at 128 bytes. At 2048 bytes, FHEStore reduces encryption latency from the CPU baseline’s 1016.276 ms to 202.442 ms, increasing the speedup to 5.02×. At 4096 bytes, the speedup reaches 5.33×.

6.3 Data Preparation Performance

This subsection compares the cost of preparing ciphertext inputs for the HPU using the CPU baseline and FHEStore. The CPU baseline

reads plaintext from the SSD and performs encoding, encryption, and ciphertext organization on the host. FHEStore combines device-side data reads with FPGA encoding and encryption, returning ciphertexts to host memory. Data-preparation latency covers SSD reads, encoding, encryption, ciphertext organization, and required data movement until ciphertexts are available in host memory.

Both paths use the same fixed-length binary records converted from TPC-H lineitem. Plaintext size B is measured in bytes per operand, corresponding to B unsigned 8-bit integers. The first operand uses the first B bytes of the record file; the second adds one to each element modulo 256. The evaluation covers bitwise OR, bitwise AND, equality, right shift, addition, subtraction, and multiplication at plaintext sizes of 64, 128, 256, 512, and 1024 bytes.

Figure 5 summarizes the stage measurements obtained under the unified-input evaluation setup described above. Panel (a) reports mean data-preparation latency relative to the mean HPU computation latency measured for the corresponding path, including completion synchronization. This normalization characterizes preparation overhead relative to downstream homomorphic computation within each path; error bars show the sample standard deviation of run-level ratios. Panel (b) directly compares data-preparation latencies: bars show the arithmetic mean of the seven operation-specific mean latencies, and error bars indicate their min-max range. The green line uses the right axis to report data-preparation speedup, defined as CPU baseline mean preparation latency divided by FHEStore mean preparation latency.

Data preparation amounts to 17.58%–57.73% of HPU computation latency for the CPU baseline and 7.29%–22.57% for FHEStore. For bitwise AND at a plaintext size of 64 bytes, the ratio decreases from 57.73% to 22.57%, indicating a substantial relative cost of data preparation for operations with short computation times. Different HPU computation latencies also produce different bar heights at the same plaintext size.

Data-preparation speedups range from 2.45× to 2.77×. At a plaintext size of 1024 bytes, mean preparation latency decreases from the CPU baseline’s 1211.67 ms to 494.50 ms for FHEStore, yielding

a 2.45 $\times$ speedup. These measurements illustrate the benefit of connecting data access, encoding, and encryption on the storage side to shorten data preparation.

6.4 Host Resource Consumption

This subsection examines how storage-side data preparation affects host CPU work and memory requirements. The CPU baseline performs encoding, encryption, and ciphertext organization on the host, whereas FHEStore generates ciphertexts on the device, with the host submitting tasks and retrieving ciphertexts. Both paths use the same SSD plaintext page and retain the resulting ciphertexts in host memory.

The experiment covers plaintext sizes of 64–4096 bytes and measures cumulative host CPU time and peak host memory over the complete execution of the host application process. Cumulative host CPU time is the sum of user and system CPU time across threads, and peak host memory is the maximum resident set size (RSS). Bar heights report means, and error bars show sample standard deviations.

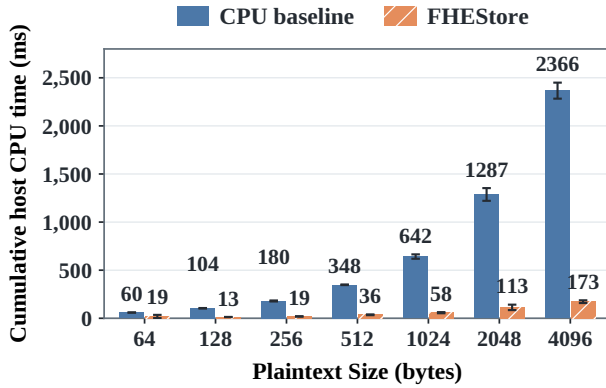


Figure 6: Cumulative host CPU time for the CPU baseline and FHEStore.

Figure 6 shows that FHEStore reduces cumulative host CPU time at every measured plaintext size. The reductions reach 91.2% and 92.7% at plaintext sizes of 2048 and 4096 bytes, respectively. At 4096 bytes, cumulative host CPU time decreases from 2366.46 ms for the CPU baseline to 172.60 ms for FHEStore. With encoding and encryption performed on the device, the host no longer executes encoding and encryption; its work shifts to task submission, ciphertext retrieval, and related management.

Figure 7 shows the change in peak host memory. At a plaintext size of 2048 bytes, FHEStore reduces peak memory from 260.09 MiB to 133.14 MiB, a 48.8% reduction; at 4096 bytes, the reduction reaches 49.4%. Both paths retain output ciphertexts in host memory, while FHEStore’s storage-side data preparation reduces the intermediate buffering required for host encoding, encryption, and ciphertext organization.

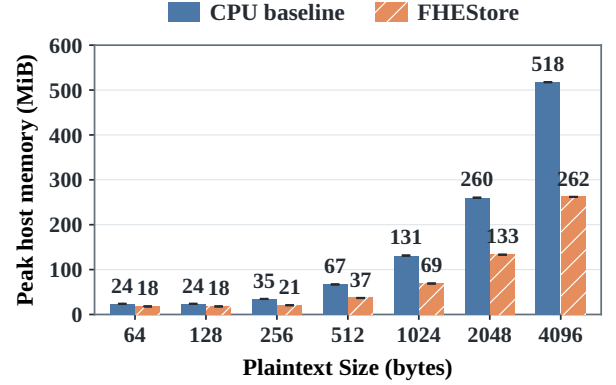


Figure 7: Peak host memory for the CPU baseline and FHEStore.

6.5 Application-Directed Data Preparation

We evaluate FHEStore’s ability to configure selection conditions and output fields for application-directed data preparation. The experiments use TPC-H lineitem and Intel Lab sensor data, with 1024 records in each dataset. Each source record is serialized into a fixed-length 512-byte binary layout for device-side preprocessing, with unused positions zero-filled. Table 4 lists six preprocessing conditions covering different value ranges, composite predicates, and encrypted output fields.

Table 4: Application-directed preprocessing conditions.

ID	SQL predicate	Encrypted field
TPC-H		
C1	WHERE extendedprice_cents <= T_{75}	quantity
C2	WHERE extendedprice_cents <= T_{50}	quantity
C3	WHERE quantity >= 20 AND shipdate < 19950101	shipmode_code
Intel Lab		
C4	WHERE humidity_pct >= H_{75}	humidity_pct
C5	WHERE humidity_pct >= H_{50}	humidity_pct
C6	WHERE temperature_milli >= 22000 AND voltage_millivolt <= 2600	moteid

The amount upper bounds T_{75} and T_{50} and humidity lower bounds H_{75} and H_{50} are selected from the corresponding field distributions to target selectivities of 75% and 50%, respectively. C3 and C6 use fixed composite predicates. Amounts are represented in cents, dates as YYYYMMDD integers, temperatures in milli-degrees Celsius, and voltages in millivolts. The SQL predicates describe the selection semantics of the tasks. These configurations vary the selected value range as well as the predicate fields, predicate combinations, and encrypted output fields.

The CPU baseline performs selection, projection, encoding, encryption, and ciphertext organization on the host. FHEStore connects these stages in a device-side processing path and generates

ciphertext only for selected field values. Data-preparation latency includes data access and processing until the ciphertexts are available in host memory. Error bars show sample standard deviations. For each condition, we verify that FHEStore and the CPU baseline select the same records and produce identical projected values.

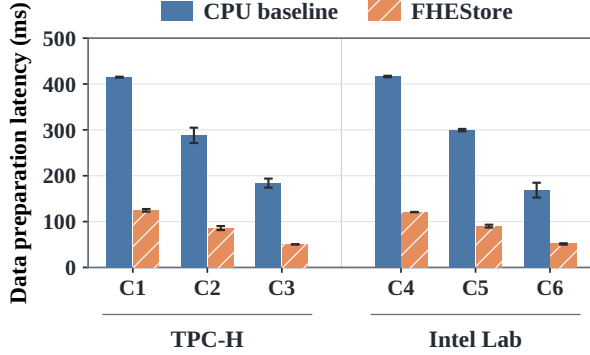


Figure 8: Data preparation under six application conditions, grouped by dataset. Error bars indicate sample standard deviations.

Figure 8 groups the six conditions by dataset. FHEStore reduces data-preparation latency in every configuration, achieving $3.29\times$ – $3.65\times$ speedups over the CPU baseline. For the composite predicate C3 on TPC-H, time decreases from 183.92 ms to 50.38 ms; for C6 on Intel Lab, it decreases from 168.62 ms to 51.23 ms, corresponding to speedups of $3.65\times$ and $3.29\times$, respectively.

Changing the selected value range also changes the encryption workload. When target selectivity decreases from 75% to 50%, FHEStore data-preparation latency decreases from 124.54 ms to 85.90 ms for TPC-H and from 120.78 ms to 90.15 ms for Intel Lab. These results demonstrate data preparation configurable by selection conditions and output fields.

6.6 Homomorphic Computation Correctness

We verify that ciphertexts generated by FHEStore support correct HPU homomorphic computation. The experiments cover bitwise OR, bitwise AND, equality, right shift, addition, subtraction, and multiplication, with plaintext sizes of 64, 128, 256, 512, and 1024 bytes per operand.

For the plaintext operands corresponding to each ciphertext batch, the host performs the same operation to generate reference results. After HPU computation, the host recovers the returned results using the corresponding key and compares the complete result sequence element by element, while also checking the output count. Addition, subtraction, and multiplication use results modulo 256, matching unsigned 8-bit arithmetic semantics.

Table 5 summarizes the reference rules and verification results, where x and y denote the plaintext operands. Each configuration is evaluated with three runs.

All 105 correctness runs match the host reference counts and element-wise values, confirming that FHEStore prepares compatible ciphertext inputs for HPU homomorphic computation.

Table 5: HPU correctness checks for FHEStore-generated ciphertexts.

Operation	Plaintext reference	Passed / Total
OR	$x \text{ OR } y$	15 / 15
AND	$x \text{ AND } y$	15 / 15
EQ	1 if $x = y$, otherwise 0	15 / 15
SHR	$x \gg (y \bmod 8)$	15 / 15
ADD	$(x + y) \bmod 256$	15 / 15
SUB	$(x - y) \bmod 256$	15 / 15
MUL	$(x \times y) \bmod 256$	15 / 15
Total	35 configurations	105 / 105

7 Related Work

Homomorphic Computation Acceleration. FHE accelerators improve homomorphic computation through specialized arithmetic and memory organization [8, 9, 28, 31]. F1 and CraterLake coordinate vector execution with data movement [18, 19], while FPT, MATCHA, and Strix optimize TFHE bootstrapping [7, 15, 24]. These designs accelerate ciphertext consumption; FHEStore addresses the preceding data-preparation stage.

Encryption and Client Processing. FPGA encryption engines have been developed for Brakerski/Fan–Vercauteren (BFV) and Cheon–Kim–Kim–Song (CKKS) [11, 12], as well as LWE and torus-based RLWE [3, 22]. Client-side systems further explore encoding, memory organization, and communication around cryptographic processing [6, 10, 25, 29]. FHEStore couples this processing to persistent-data access.

Computational Storage. Computational storage systems connect persistent data to programmable processing. Ibex offloads selection, projection, and aggregation [27], and Biscuit composes host and device tasks through ordered data ports [5]. INSIDER and DONGLE 2.0 provide programmable interfaces for data access and FPGA execution [17, 26]. FHEStore applies operator stream composition to data preparation while accounting for ciphertext expansion.

Storage-Coupled FHE Evaluation. Storage-coupled FHE designs integrate storage access with homomorphic evaluation [14, 21]. Ohba et al. convey ciphertexts, keys, and programs through NVMe to an FPGA TFHE accelerator, while Suzuki and Yamana study CSD-coupled homomorphic execution. FHEStore instead prepares ciphertexts from persistent plaintext for external homomorphic computation.

8 Conclusion

FHEStore uses FPGA-based computational storage for FHE data preparation from persistent data. Its HLS operator integrates radix encoding and TFHE encryption; each generated mask coefficient feeds dot-product accumulation and incremental ciphertext output. Operator stream composition connects optional preprocessing and encryption, accounting for ciphertext expansion. For 1 KiB plaintext, the encryption operator and data preparation achieve $5.00\times$ and $2.45\times$ speedups over host CPU baselines, with lower cumulative host CPU time and peak host memory. Generated ciphertexts support correct HPU homomorphic computation.

References

- [1] Peter Bodik, Wei Hong, Carlos Guestrin, Samuel Madden, Mark Paskin, and Romain Thibaux. 2004. Intel Lab Data. <https://db.csail.mit.edu/labdata/labdata.html>
- [2] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology* 33 (2020), 34–91. doi:10.1007/s00145-019-09319-x
- [3] Najmi Azzahra Feryputri, Infall Syafalni, Nana Sutisna, Nur Ahmadi, Rahmat Mulyawan, and Trio Adiono. 2025. FPGA-Based Hardware Accelerator for LWE Encryption and Decryption with TFHE Scheme. In *2025 IEEE 38th International System-on-Chip Conference (SOCC)*. IEEE, 1–6. doi:10.1109/SOCC66126.2025.11235379
- [4] Craig Gentry. 2009. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing (STOC '09)*. Association for Computing Machinery, New York, NY, USA, 169–178. doi:10.1145/1536414.1536440
- [5] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. 2016. Biscuit: A Framework for Near-Data Processing of Big Data Workloads. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 153–165. doi:10.1109/ISCA.2016.23
- [6] Saransh Gupta, Rosario Cammarota, and Tajana Šimunić. 2024. MemFHE: End-to-end Computing with Fully Homomorphic Encryption in Memory. *ACM Transactions on Embedded Computing Systems* 23, 2, Article 28 (2024), 23 pages. doi:10.1145/3569955
- [7] Lei Jiang, Qian Lou, and Nrushad Joshi. 2022. MATCHA: A Fast and Energy-Efficient Accelerator for Fully Homomorphic Encryption over the Torus. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC '22)*. Association for Computing Machinery, New York, NY, USA, 235–240. doi:10.1145/3489517.3530435
- [8] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2022. ARK: Fully Homomorphic Encryption Accelerator with Runtime Data Generation and Inter-Operation Key Reuse. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1237–1254. doi:10.1109/MICRO56248.2022.00086
- [9] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. BTS: An Accelerator for Bootstrappable Fully Homomorphic Encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 711–725. doi:10.1145/3470496.3527415
- [10] Florian Krieger, Florian Hirner, Ahmet Can Mert, and Sujoy Sinha Roy. 2024. Aloha-HE: A Low-Area Hardware Accelerator for Client-Side Operations in Homomorphic Encryption. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6. doi:10.23919/DATE58400.2024.10546608
- [11] Jaehyeok Lee, Phap Ngoc Duong, and Hanho Lee. 2023. Configurable Encryption and Decryption Architectures for CKKS-Based Homomorphic Encryption. *Sensors* 23, 17 (2023), 7389. doi:10.3390/s23177389
- [12] Ahmet Can Mert, Erdiç Öztürk, and Erkan Savaş. 2020. Design and Implementation of Encryption/Decryption Architectures for BFV Homomorphic Encryption Scheme. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, 2 (2020), 353–362. doi:10.1109/TVLSI.2019.2943127
- [13] NVM Express. 2025. *NVM Express Subsystem Local Memory Command Set Specification* (revision 1.2 ed.). NVM Express, Inc. <https://nvmexpress.org/wp-content/uploads/NVM-Express-Subsystem-Local-Memory-Command-Set-Specification-Revision-1.2-2025.08.01-Ratified.pdf>
- [14] Yoshihiro Ohba, Tomoya Sanuki, Claude Gravel, Kentaro Mihara, Asuka Waka-sugi, and Kenta Adachi. 2025. An NVMe-Based Secure Computing Platform With FPGA-Based TFHE Accelerator. *IEEE Access* 13 (2025), 69980–69997. doi:10.1109/ACCESS.2025.3561728
- [15] Adiweni Putra, Prasetyo, Yi Chen, John Kim, and Joo-Young Kim. 2023. Strix: An End-to-End Streaming Architecture with Two-Level Ciphertext Batching for Fully Homomorphic Encryption with Programmable Bootstrapping. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 1319–1331. doi:10.1145/3613424.3614264
- [16] Oded Regev. 2009. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. *J. ACM* 56, 6, Article 34 (Sept. 2009), 40 pages. doi:10.1145/1568318.1568324
- [17] Zhenyuan Ruan, Tong He, and Jason Cong. 2019. INSIDER: Designing In-Storage Computing System for Emerging High-Performance Drive. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 379–394. <https://www.usenix.org/conference/atc19/presentation/ruan>
- [18] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/3466752.3480070
- [19] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. 2022. CraterLake: A Hardware Accelerator for Efficient Unbounded Computation on Encrypted Data. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 173–187. doi:10.1145/3470496.3527393
- [20] SNIA. 2025. *Computational Storage Architecture and Programming Model* (version 1.1 ed.). SNIA. <https://www.snia.org/sites/default/files/technical-work/computational/release/SNIA-Computational-Storage-Architecture-and-Programming-Model-1.1.pdf> Released May 31, 2025.
- [21] Takuya Suzuki and Hayato Yamana. 2023. Designing In-Storage Computing for Low Latency and High Throughput Homomorphic Encrypted Execution. In *2023 IEEE 8th International Conference on Big Data Analytics (ICBDA)*. IEEE, 127–136. doi:10.1109/ICBDA57405.2023.10104970
- [22] Infall Syafalni, Najmi Azzahra Feryputri, Nana Sutisna, Nur Ahmadi, Rahmat Mulyawan, Tutun Juhana, and Trio Adiono. 2026. Multicore Encryption-Decryption Hardware Accelerator for Torus-Based RLWE FHE. In *2026 IEEE Symposium on Low-Power and High-Speed Chips and Systems (COOL CHIPS)*. IEEE, 1–3. doi:10.1109/COOLCHIPS68842.2026.11556998
- [23] Transaction Processing Performance Council. 2022. *TPC Benchmark H Standard Specification* (revision 3.0.1 ed.). Transaction Processing Performance Council. https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H_v3.0.1.pdf
- [24] Michiel Van Beirendonck, Jan-Pieter D'Anvers, Furkan Turan, and Ingrid Verbauwhede. 2023. FPT: A Fixed-Point Accelerator for Torus Fully Homomorphic Encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 741–755. doi:10.1145/3576915.3623159
- [25] McKenzie van der Hagen and Brandon Lucia. 2022. Client-Optimized Algorithms and Acceleration for Encrypted Compute Offloading. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 683–696. doi:10.1145/3503222.3507737
- [26] Linus Y. Wong, Jialiing Zhang, and Jing (Jane) Li. 2024. DONGLE 2.0: Direct FPGA-Orchestrated NVMe Storage for HLS. *ACM Transactions on Reconfigurable Technology and Systems* 17, 3, Article 45 (2024), 32 pages. doi:10.1145/3650038
- [27] Louis Woods, Zolt István, and Gustavo Alonso. 2014. Ibex—An Intelligent Storage Engine with Support for Advanced SQL Off-loading. *Proceedings of the VLDB Endowment* 7, 11 (2014), 963–974. doi:10.14778/2732967.2732972
- [28] Zhihan Xu, Rajgopal Kannan, and Viktor Prasanna. 2026. HERA: A Bandwidth-efficient Accelerator for Fully Homomorphic Encryption on HBM-enabled FPGA. In *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '26)*. Association for Computing Machinery, New York, NY, USA, 265–276. doi:10.1145/3748173.3779201
- [29] Sungwoong Yune, Hyejeong Lee, Adiweni Putra, Hyunjun Cho, Cuong Duong Manh, Jaeho Jeon, and Joo-Young Kim. 2025. ABC-FHE: A Resource-Efficient Accelerator Enabling Bootstrappable Parameters for Client-Side Fully Homomorphic Encryption. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7. doi:10.1109/DAC63849.2025.11132592
- [30] Zama. 2025. A systemVerilog implementation of the Homomorphic Processing Unit (HPU) targeting AMD Alveo V80 FPGA board. https://github.com/zama-ai/hpu_fpga
- [31] Minxuan Zhou, Yujin Nam, Pranav Gangwar, Weihong Xu, Arpan Dutta, Kartikeyan Subramanyam, Chris Wilkerson, Rosario Cammarota, Saransh Gupta, and Tajana Rosing. 2023. FHEmem: A Processing In-Memory Accelerator for Fully Homomorphic Encryption. arXiv preprint arXiv:2311.16293. <https://arxiv.org/abs/2311.16293v1>